Métodos de Ordenação

- O que é ordenar: organizar uma estrutura de dados (cresc ou decresc) usando uma ou mais chaves de controles (variável)
- Por que ordenar? porque estruturas ordenadas tem a busca ou a pesquisa mais eficiente, devido aos algoritmos baseados em árvores

Classificação, complexidade e aplicações

Este documento reúne os principais métodos de ordenação, classificados quanto ao uso de memória (interna ou externa), estabilidade, complexidade de tempo, porção do vetor que fica ordenada durante o processo, exemplos de aplicação e frequência de uso na prática.

Tabela comparativa

Método	Memória	Estabilidade	Complexidade (melhor/médio/pior)	Porção ordenada
Bolha (Bubble sort)	Interna	Estável	O(n) / O(n²) / O(n²)	Final (maiores ao fim)
Seleção (Selection sort)	Interna	Instável	O(n²) / O(n²) / O(n²)	Início
Inserção (Insertion sort)	Interna	Estável	O(n) / O(n²) / O(n²)	Início
Pente (Comb sort)	Interna	Instável	O(n log n) / O(n²/2ᵖ) / O(n²)	Tende ao final (gaps)
Agitação (Cocktail/ Shaker)	Interna	Estável	O(n) / O(n²) / O(n²)	Ambas extremidades
Shellsort	Interna	Instável	O(n log n) / ~O(n^1.25) / O(n²)*	Sem porção fixa (por gaps)
Bucketsort (Balde)	Interna (auxiliar)	Estável**	O(n+k) / O(n+k) / O(n²)	Distribuição por baldes
Radix sort	Interna (auxiliar)	Estável	O(nk) / O(nk) / O(nk)	Distribuição por dígitos
Heapsort	Interna	Instável	O(n log n) / O(n log n) / O(n log n)	Final (extrai o máximo)
Mergesort	Interna e Externa	Estável	O(n log n) / O(n log n) / O(n log n)	Mesclagem (dividir e conquistar)
Quicksort	Interna	Instável	O(n log n) / O(n log n) / O(n²)	Particionamento (pivô)

Conceitos de classificação

Memória interna vs. externa: um algoritmo é de memória interna quando todos os dados cabem na memória principal (RAM) durante a ordenação. É de memória externa quando o volume de dados excede a RAM e parte permanece em memória secundária (disco), sendo lida em blocos. A maioria dos métodos abaixo é de ordenação interna; o Mergesort é o algoritmo classicamente adaptado para ordenação externa (external merge sort). Bucket e Radix são internos, mas não são in-place: usam memória auxiliar proporcional aos dados.

Estabilidade: um método é estável quando preserva a ordem relativa de elementos com chaves iguais. Isso é importante ao ordenar por múltiplos critérios em sequência.

Complexidade: o número de operações em função de n. Analisada nos casos melhor, médio e pior. Métodos O(n²) são adequados a conjuntos pequenos; métodos O(n log n) escalam bem.

Porção ordenada: durante a execução, parte do vetor já está ordenada. Saber onde essa porção cresce (início, fim ou ambos) ajuda a entender e depurar cada método.

Descrição de cada método

Bolha (Bubble sort)

Classificação: Memória interna (in-place) · Estável · O(n²) · porção ordenada cresce no final.

Exemplos de aplicação: Ensino de lógica de ordenação; conjuntos muito pequenos ou quase ordenados (a versão otimizada detecta vetor já ordenado em O(n)).

Uso comum: Raro na prática pela baixa eficiência; valor principalmente didático.

Seleção (Selection sort)

Classificação: Memória interna (in-place) · Instável · O(n²) em todos os casos · porção ordenada cresce no início.

Exemplos de aplicação: Situações em que o custo de escrita/troca é alto, pois faz no máximo O(n) trocas; conjuntos pequenos.

Uso comum: Pouco usado; didático. Útil quando minimizar o número de trocas é prioridade.

Inserção (Insertion sort)

Classificação: Memória interna (in-place) · Estável · $O(n^2)$ ($O(n)$ no melhor caso) · porção ordenada cresce no início.

Exemplos de aplicação: Listas pequenas ou quase ordenadas; ordenação online (elementos chegando aos poucos).

Uso comum: Muito usado como sub-rotina: Timsort e Introsort recorrem à inserção para partições pequenas.

Pente (Comb sort)

Classificação: Memória interna (in-place) · Instável · pior caso $O(n^2)$, médio próximo de $O(n \log n)$ · melhoria do Bubble usando um gap decrescente (fator ~1,3).

Exemplos de aplicação: Alternativa simples ao Bubble quando se quer melhor desempenho sem grande complexidade de código. **Uso comum: Pouco usado; nicho educacional e implementações simples.**

Agitação (Cocktail / Shaker sort)

Classificação: Memória interna (in-place) · Estável · $O(n^2)$ ($O(n)$ no melhor caso) · ordena nas duas extremidades (bidirecional).

Exemplos de aplicação: Variação do Bubble que percorre o vetor nos dois sentidos; bom para vetores quase ordenados com poucos elementos fora de posição. **Uso comum: Pouco usado na prática; principalmente didático.**

Shellsort

Classificação: Memória interna (in-place) · Instável · complexidade depende da sequência de gaps (de $O(n^2)$ a $\sim O(n \log^2 n)$) · ordena por incrementos (h-sorting), sem porção fixa.

Exemplos de aplicação: Sistemas embarcados e bibliotecas onde se evita recursão e memória extra (ex.: uClibc); conjuntos de tamanho moderado. **Uso comum: Uso moderado; boa escolha quando código simples, in-place e sem recursão é desejável.**

Bucketsort (Balde)

Classificação: Memória interna com memória auxiliar (não in-place) · Estável (se a ordenação interna dos baldes for estável) · $O(n+k)$ médio, $O(n^2)$ pior · algoritmo de distribuição.

Exemplos de aplicação: Dados numéricos uniformemente distribuídos, como números reais em [0,1); pré-processamento de grandes volumes homogêneos. Uso comum: Uso específico, quando a distribuição das chaves é conhecida e uniforme.

Radix sort

Classificação: Memória interna com memória auxiliar (não in-place) · Estável (versão LSD) · $O(n \cdot k)$, sem comparações entre chaves · distribuição por dígitos/blocos.

Exemplos de aplicação: Inteiros, strings de tamanho fixo, CEPs, identificadores; ordenação de chaves com poucos dígitos.

Uso comum: Usado quando as chaves são inteiras ou curtas; comum em bancos de dados e processamento de grandes volumes.

Heapsort

Classificação: Memória interna (in-place) · Instável · $O(n \log n)$ em todos os casos · porção ordenada cresce no final (extrai o máximo do heap).

Exemplos de aplicação: Quando se exige pior caso garantido $O(n \log n)$ e espaço $O(1)$; sistemas de tempo real e embarcados. Uso comum: Uso moderado; serve de fallback do Introsort (std::sort do C++) para evitar o pior caso do Quicksort.

Mergesort

Classificação: Interna e Externa · Estável · $O(n \log n)$ em todos os casos · dividir e conquistar com mesclagem; usa $O(n)$ de memória auxiliar.

Exemplos de aplicação: Ordenação externa de arquivos grandes (external merge sort); listas ligadas; quando estabilidade é obrigatória.

Uso comum: Muito usado; base do Timsort (Python, Java) e do Collections.sort do Java para objetos.

Quicksort

Classificação: Memória interna (in-place, $O(\log n)$ de pilha) · Instável · $O(n \log n)$ médio, $O(n^2)$ pior · dividir e conquistar por particionamento em torno de um pivô.

Exemplos de aplicação: Ordenação de arrays de propósito geral; muito eficiente na média por boa localidade de cache.

Uso comum: É o método mais usado na prática para arrays: base do qsort da C e do Introsort (std::sort do C++).

Método estável x instável

Um método é estável quando dois elementos de mesma chave mantêm, no resultado, a mesma ordem relativa que tinham na entrada. Num método instável, essa ordem pode ser trocada.

Por que importa: ao ordenar por vários critérios em sequência (por exemplo, ordenar uma lista de alunos primeiro por nome e depois por nota), a estabilidade preserva a ordenação anterior dentro dos empates. Com um método instável, a primeira ordenação pode ser desfeita nos elementos empatados.

Estáveis: Bolha, Inserção, Agitação, Mergesort, Radix e Bucketsort (quando o balde é ordenado de forma estável).

Instáveis: Seleção, Pente, Shellsort, Heapsort e Quicksort.

Dica: um método instável pode ser tornado estável artificialmente incluindo o índice original como critério de desempate — ou seja, quando as chaves são iguais, compara-se a posição inicial. Isso custa memória extra e uma comparação adicional.

Codificação simples x complexa

Os métodos diferem muito na facilidade de implementação. Isso conta na escolha: para conjuntos pequenos, um algoritmo simples e correto costuma ser preferível a um sofisticado e sujeito a erros.

Codificação simples: Bolha, Seleção e Inserção — poucas linhas, dois laços aninhados, sem recursão nem estruturas auxiliares. São os mais fáceis de escrever e depurar, por isso são os primeiros ensinados.

Codificação intermediária: Agitação, Pente e Shellsort — pequenas variações dos anteriores (mudança de sentido ou de gap), ainda simples.

Codificação mais complexa: Quicksort, Mergesort, Heapsort, Radix e Bucketsort — exigem recursão, particionamento, mesclagem, estrutura de heap ou baldes auxiliares. São mais rápidos, mas mais trabalhosos e mais propensos a erros de implementação (índices, casos-limite, estouro de pilha).

Alterando a forma de comparação (compareTo / Comparator)

Os métodos baseados em comparação (todos, exceto Radix e Bucket, que distribuem) não dependem do tipo de dado: eles só precisam saber, dados dois elementos a e b, se a vem antes, depois ou é igual a b. Essa decisão é isolada em um único ponto — o método de comparação.

Em Java, por exemplo, os objetos implementam compareTo (interface Comparable) ou recebem um Comparator. O algoritmo de ordenação chama sempre a.compareTo(b) e ordena conforme o sinal retornado (negativo, zero ou positivo). Para mudar o critério — de crescente para decrescente, ou de ordenar por nome para ordenar por idade — basta reescrever o compareTo (ou passar outro Comparator); o código do algoritmo não muda.

Crescente x decrescente: retornar b - a em vez de a - b (ou inverter a condição de troca) inverte a ordenação.

Ordenar por outro campo / múltiplos critérios: comparar primeiro um campo e, em empate, comparar o próximo (por exemplo, sobrenome e depois nome). É exatamente onde a estabilidade ajuda.

Regra prática: separe a política de comparação (o compareTo) do mecanismo de ordenação (o algoritmo). Assim você troca o critério sem tocar no algoritmo, e reaproveita o mesmo algoritmo para qualquer tipo de dado.

Complexidade e a relação entre trocas e comparações

A complexidade de um método de ordenação é medida contando as duas operações dominantes: comparações (quantas vezes dois elementos são comparados entre si) e trocas/movimentações (quantas vezes um elemento muda de posição). Conta-se quantas dessas operações ocorrem em função de n (o número de elementos) e mantém-se apenas o termo que cresce mais rápido — esse termo é a ordem do algoritmo (notação O).

De onde vem cada ordem

$O(n^2)$ — laços aninhados: métodos com dois laços aninhados sobre o vetor (Bolha, Seleção, Inserção): o laço interno executa cerca de n vezes para cada uma das n voltas do laço externo, dando aproximadamente $n \times n / 2$ comparações → $O(n^2)$.

$O(n \log n)$ — dividir para conquistar: métodos que dividem o problema pela metade a cada passo (Mergesort, Quicksort, Heapsort): há cerca de log n níveis e cada nível faz O(n) trabalho → $O(n \log n)$.

O(n) aproximado — distribuição: Radix e Bucket não comparam elementos entre si; distribuem por dígito/faixa, atingindo $O(n \cdot k)$ ou $O(n+k)$. Por isso escapam do limite de $O(n \log n)$ dos métodos por comparação.

A relação: comparações nem sempre acompanham as trocas

Dois métodos podem ter a mesma ordem geral e ainda assim se comportar de forma bem diferente, porque comparar e trocar são operações separadas e com custos separados. Comparar costuma ser barato; trocar/mover pode ser caro (quando os elementos são grandes ou estão em disco). A tabela abaixo mostra o pior caso de cada operação:

Método	Comparações	Trocas / movimentações	Ordem geral
Seleção	$O(n^2)$	$O(n)$	$O(n^2)$
Bolha	$O(n^2)$	$O(n^2)$	$O(n^2)$
Inserção	$O(n^2)$	$O(n^2)$	$O(n^2)$
Quicksort	$O(n \log n)$ méd.	$O(n \log n)$ méd.	$O(n \log n)$ méd.
Mergesort	$O(n \log n)$	$O(n \log n)$ mov.	$O(n \log n)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Comparações e trocas medidas no pior caso; "méd." indica caso médio (o pior caso do Quicksort é $O(n^2)$).

Lendo a tabela, três casos ilustram bem a relação:

Seleção: faz muitas comparações — $O(n^2)$ — mas só $O(n)$ trocas, pois coloca cada elemento direto na posição final (uma troca por volta). É a escolha quando mover dados é caro.

Bolha: faz $O(n^2)$ comparações E $O(n^2)$ trocas, porque troca vizinhos repetidamente a cada passagem. Por isso costuma ser o mais lento dos três na prática, apesar da mesma ordem.

Inserção: comparações e movimentações crescem juntas — $O(n^2)$ no pior caso —, mas caem para $O(n)$ quando o vetor já está quase ordenado, pois quase não desloca elementos. Daí ser rápido em entradas pequenas ou parcialmente ordenadas.

Como determinar na prática

Para descobrir a ordem de um método sem decorar fórmulas, instrumente o código com dois contadores — um para comparações (incrementado a cada compareTo) e outro para trocas — e rode com tamanhos de entrada crescentes, dobrando n a cada teste. O padrão de crescimento revela a ordem:

Se for $O(n^2)$: ao dobrar n, a contagem quadruplica ($2^2 = 4$).

Se for $O(n \log n)$: ao dobrar n, a contagem pouco mais que dobra.

Se for $O(n)$: ao dobrar n, a contagem apenas dobra.

Resumo: o número de comparações define o "esforço de decidir a ordem" e o número de trocas define o "esforço de reorganizar". A complexidade final é dominada pela maior das duas; por isso, ao escolher um método, olhe as duas contagens — e não só a ordem geral — considerando se comparar ou mover é o mais caro no seu caso.

```
def bolha(lista):

    houve_troca = true

    qtd_comparacoes = 0

    qtd_trocas = 0

    while (houve_troca):

        houveTroca = false

        for i in range (len(lista) - 1):

            qtd_comparacoes+=1

            if (lista[i] > lista[i+1]):

                qtd_trocas+=1

                houve_troca = true

                tmp = lista[i]

                lista[i] = lista[i+1]

                lista[i+1] = tmp
```

Método mais utilizado na prática

Não existe um único método universalmente melhor — a escolha depende do contexto. Na prática, os mais empregados são:

Quicksort / Introsort: o mais usado para ordenar arrays em memória, por seu excelente desempenho médio e boa localidade de cache. Aparece no qsort da linguagem C e, combinado com Heapsort e Inserção, forma o Introsort do std::sort (C++).

Mergesort / Timsort: preferido quando se exige estabilidade e pior caso garantido O(n log n). Deriva o Timsort, algoritmo padrão de Python (sorted/list.sort) e do Java para objetos, e é a base da ordenação externa de grandes volumes.

Inserção: usado como sub-rotina dentro dos anteriores para partições pequenas, onde seu baixo custo constante o torna mais rápido que os métodos O(n log n).

Resumo: Quicksort domina a ordenação de arrays; Mergesort/Timsort domina quando se precisa de estabilidade ou ordenação externa; os métodos $O(n^2)$ (bolha, seleção, inserção) permanecem relevantes para conjuntos pequenos e para fins didáticos.